OPENWRT
CYBERSECURITY HOMELAB

KVM Virtualisation • Firewall Hardening • WireGuard VPN • Suricata IDS

Author
OTHMANE EL MQIDDEM

github.com/oeisthename/homelab-openwrt-security

Contents

List of Acronyms . . . **3**

1 Executive Summary . . . **5**

2 Engineering Philosophy . . . **5**
- 2.1 Risk-First Architecture . . . 5
- 2.2 Principle of Least Privilege . . . 6
- 2.3 Document Everything, Especially Failures . . . 6
- 2.4 Measure All Security Outcomes . . . 6
- 2.5 Snapshot-Driven Iteration . . . 6

3 Hardware, Software & Architecture . . . **7**
- 3.1 Hardware Inventory . . . 7
- 3.2 Software Stack . . . 7
- 3.3 Network Topology . . . 7
- 3.4 IP Address Schema . . . 8

4 Session 1 — KVM Setup & OpenWRT VM Deployment . . . **8**
- 4.1 Hardware Compatibility Research . . . 8
- 4.2 KVM Virtualisation Stack . . . 8
- 4.3 OpenWRT Image Preparation . . . 9
- 4.4 VM Creation — Issue Log . . . 9
- 4.5 OpenWRT First Boot . . . 11
- 4.6 Session 1 Snapshots . . . 11

5 Session 2 — Recovery, LuCI, SSH Hardening & Nmap . . . **12**
- 5.1 Issue 6 — VM Definition Lost After Host Reboot . . . 12
- 5.2 Issue 7 — Image Permission Denied . . . 12
- 5.3 Internet Connectivity Verification . . . 12
- 5.4 LuCI Web Interface Access . . . 13
- 5.5 SSH Key Authentication Setup . . . 14
- 5.6 Nmap Attack Surface Analysis . . . 15
- 5.7 Firewall Hardening — Source IP Restriction . . . 16
- 5.8 VM Persistence Fix . . . 16
- 5.9 Session 2 Snapshots . . . 16
- 5.10 Session 2 Hardening Summary . . . 16

6 Session 3 — Firewall Audit & WireGuard VPN . . . **17**
- 6.1 Session Startup . . . 17
- 6.2 Firewall Ruleset Audit . . . 17
- 6.3 WireGuard VPN Deployment . . . 18
- 6.4 Session 3 Snapshot . . . 20

7 Session 4 — Suricata IDS & GitHub Publication . . . **21**
- 7.1 Session Startup — Three-Command Workflow . . . 21
- 7.2 Suricata IDS Overview . . . 21
- 7.3 Suricata Setup . . . 21
- 7.4 Custom Rule Writing . . . 22
- 7.5 Alert Triggering & Verification . . . 23
- 7.6 GitHub Repository Publication . . . 23
- 7.7 Session 4 Snapshot . . . 24

8 Complete Issues & Troubleshooting Log . . . **24**

9 Security Controls — Final State . . . **24**
- 9.1 Attack Surface Evolution . . . 24

10 Performance Metrics . . . **25**

11 Cost Analysis . . . **25**

12 Snapshot History **25**

13 Skills & Competencies Demonstrated **25**

14 Quick Reference Commands **25**
14.1 Lab Start / Stop 26
14.2 VM Management 26
14.3 OpenWRT UCI 26
14.4 WireGuard 26
14.5 Suricata IDS 26

15 Future Roadmap **26**

List of Acronyms

ACL	Access Control List
AEAD	Authenticated Encryption with Associated Data
ADSL	Asymmetric Digital Subscriber Line
AF_PACKET	Address Family Packet — Linux zero-copy socket API
API	Application Programming Interface
DHCP	Dynamic Host Configuration Protocol
DMZ	Demilitarized Zone
DNS	Domain Name System
ECC	Elliptic Curve Cryptography
ET	Emerging Threats (open-source Suricata/Snort ruleset)
GPL	GNU General Public License
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IGMP	Internet Group Management Protocol
IPS	Intrusion Prevention System
IPSec	Internet Protocol Security
ISP	Internet Service Provider
KVM	Kernel-based Virtual Machine
LAN	Local Area Network
LuCI	Lua Configuration Interface — OpenWRT web admin panel
MAC	Media Access Control address
MLD	Multicast Listener Discovery
MTU	Maximum Transmission Unit
NAT	Network Address Translation
NIC	Network Interface Card
nftables	Netfilter Tables — Linux kernel packet filtering
Nmap	Network Mapper
OISF	Open Information Security Foundation
opkg	OpenWRT Package Manager
PoLP	Principle of Least Privilege
QEMU	Quick EMUlator
qcow2	QEMU Copy-On-Write v2 disk image format
RFC	Request for Comments
RSA	Rivest–Shamir–Adleman public-key algorithm
SHA256	Secure Hash Algorithm 256-bit
SIEM	Security Information and Event Management
SID	Signature ID
SOC	Security Operations Center
SSH	Secure Shell
SYN	Synchronise TCP flag
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TUN	Network Tunnel virtual interface
UCI	Unified Configuration Interface
UDP	User Datagram Protocol
VLAN	Virtual Local Area Network

VM	Virtual Machine
VNC	Virtual Network Computing
VPN	Virtual Private Network
WAN	Wide Area Network
WireGuard	WireGuard VPN Protocol (Linux kernel $\geq$5.6)

1. Executive Summary

This document is the complete engineering record of the OpenWRT Cybersecurity Homelab project, a from-scratch security lab conceived, built, and hardened across four structured sessions on commodity hardware. It was designed from the outset not merely as a functional lab environment, but as a rigorous engineering exercise: every decision is justified, every command is reproduced exactly as it was executed, every error is documented with its precise root cause rather than just its symptom, and every fix is explained alongside the underlying security or systems concept it illustrates.

The project spans the full lifecycle of a security-focused infrastructure build, from initial hardware compatibility research and a deliberate architecture pivot, through hypervisor setup, virtual machine provisioning, network configuration, multi-layer firewall hardening, encrypted VPN deployment, and live intrusion detection. Nothing was skipped and nothing was sanitised after the fact. The eight issues encountered during the build are documented in the same detail as the successful outcomes, because failure analysis is where the most durable technical understanding is formed.

The intended audience is anyone who wants to understand not just what was built, but how engineering decisions were made under real constraints, how problems were diagnosed systematically, and how security principles were applied at every layer of the stack, from file permissions and setuid binaries at the OS level, up to cryptographic key management and network traffic analysis. Whether you are an IT professional looking to build a comparable environment, a recruiter evaluating practical depth, or a student working through the same material independently, this document is intended to give you the full picture.

Attribute	Value
Platform	OpenWRT 23.05.3 (x86/64) on KVM/QEMU
Host	HP EliteBook 850 G3 — Ubuntu Linux (user: kairos)
VPN Protocol	WireGuard — ChaCha20-Poly1305 AEAD, Curve25519
IDS Engine	Suricata 7.0.3 — 48,872 signatures + 3 custom rules
Attack Surface	2 open TCP ports out of 65,535 scanned
Issues Resolved	8 — each with root cause analysis and targeted fix
Snapshots	6 named rollback points at major milestones
Repository	github.com/oeisthename/homelab-openwrt-security

2. Engineering Philosophy

2.1 Risk-First Architecture

Before writing a single command or downloading a single file, the target hardware was assessed formally against the OpenWRT Table of Hardware. This step is often skipped by engineers eager to start executing but it is one of the most consequential decisions in any infrastructure project, because the cost of proceeding on a false assumption compounds with every hour of work built on top of it.

The TP-Link TD-W8961N Ver:3.0 was the original target device. Initial research returned no entry in the OpenWRT ToH for that specific version. Deeper investigation revealed a structural problem that goes beyond a simple compatibility gap: the device is an ADSL modem-router, meaning it contains two distinct subsystems a router/WiFi SoC that runs the operating system, and a DSL modem chip that handles the physical phone line connection. OpenWRT can potentially support the router SoC, but the DSL modem chip uses proprietary firmware with no open-source driver support. Even a successful flash of the router partition would leave the DSL subsystem non-functional, rendering the device unable to connect to the internet and therefore useless as a lab gateway. Compounding this, the device has no accessible serial console, which means a failed flash has no recovery path short of hardware-level soldering.

The risk profile was clear: high probability of permanent hardware damage, low probability of a usable outcome, and no rollback option. Proceeding would have been an engineering error, not a bold experiment.

The decision was made to pivot the entire architecture to KVM virtualisation on the existing host machine. This was not a compromise, it was a strictly superior outcome. KVM produces identical networking and firewall learning experiences while adding capabilities that physical hardware fundamentally cannot provide: named snapshots with instant rollback, multiple simultaneous VMs for future attacker/victim scenarios, zero hardware risk on every configuration change, and direct relevance to how production infrastructure actually operates at scale. AWS, Google Cloud Platform, and most enterprise private cloud environments run on KVM. Building fluency with it is not a workaround; it is the skill itself.

The broader principle this decision encodes is that risk-first thinking must precede execution thinking. Identifying a high-risk, low-reward path early and engineering a better path around it, is a more valuable demonstration of professional judgement than successfully executing a risky plan.

TIP: Portfolio Signal

Recognising a high-risk/low-reward path and engineering around it is a senior-level skill.

2.2 Principle of Least Privilege

The Principle of Least Privilege is not a configuration option to be enabled at the end of a project, it is a design constraint applied from the first command. Every component in this lab was given exactly the access it needs to function, and nothing more.

In practice this meant operating as the regular user `kairos` throughout, invoking `sudo` only for the specific operations that genuinely require elevated privileges rather than opening a root shell and leaving it open. It meant accepting that QEMU deliberately runs as the unprivileged system user `libvirt-qemu` a design choice that limits the blast radius of any QEMU vulnerability to that restricted account rather than the entire host. It meant writing firewall rules that specify not just which port is allowed, but which exact source IP is permitted to reach it, so that SSH and LuCI are accessible only from `192.168.122.1` and invisible to every other host on the network. It meant disabling every service, protocol, and firewall rule that was not actively required, rather than leaving them enabled on the assumption they would probably never be exploited.

Each of these individually is a small constraint. Composed across every layer of the stack, they mean that compromising any single component yields an attacker the minimum possible foothold, which is the precise goal of defence in depth.

2.3 Document Everything, Especially Failures

Eight real issues were encountered across the four sessions, ranging from a libvirt database synchronisation problem and a missing QEMU bridge ACL file, to a silent Suricata rule load failure caused by a single misplaced space in a YAML configuration. None of them were edited out or glossed over in retrospect. Each is documented with the exact error message as it appeared in the terminal, the root cause analysis that explains why it happened rather than just what happened, the minimal targeted fix that resolved it, and the security or systems concept the issue brought into focus. This level of documentation is a deliberate choice rooted in a simple observation: a lab environment where everything works on the first attempt teaches you how to follow instructions. A lab where things break and where you are forced to reason carefully about why, teaches you how to think. The failures in this project are not embarrassing footnotes; they are the most technically instructive parts of the entire record, and they are presented with the same rigour as every successful outcome

2.4 Measure All Security Outcomes

Security work that cannot be expressed in numbers is difficult to defend, difficult to compare, and difficult to improve. Every meaningful outcome in this project was therefore measured and stated precisely. The attack surface was quantified by running a full 65,535-port Nmap scan against the router's WAN interface and counting the result: 2 open ports. Not "minimal exposure" or "well hardened" 2 open ports, from 65,535 scanned. The firewall ruleset was audited rule by rule, and the result was recorded as a before-and-after count: 11 rules reduced to 7, with each of the 4 removed rules documented alongside the specific reason it was unnecessary. IDS coverage was not described as "comprehensive" it was stated as 48,872 active signatures loaded from the Emerging Threats Open ruleset, plus 3 custom rules written for this environment. VPN encryption overhead was not dismissed as negligible it was measured at an average round-trip latency of 1.148 ms through the WireGuard tunnel, confirming that the performance cost of ChaCha20-Poly1305 encryption at this scale is effectively zero. This habit of measurement matters beyond the numbers themselves. It forces clarity about what has actually been achieved, it creates a baseline that makes future regressions detectable, and it produces the kind of concrete, verifiable claims that distinguish engineering documentation from opinion.

2.5 Snapshot-Driven Iteration

Six named snapshots were created at meaningful milestones. Each is a reversible state enabling fearless experimentation. Every significant configuration change was preceded by a snapshot.

3. Hardware, Software & Architecture

3.1 Hardware Inventory

Component	Model / Spec	Role
Laptop (Host)	HP EliteBook 850 G3, Intel VT-x, 8 threads	KVM hypervisor — runs all VMs
Host OS	Ubuntu Linux — user: kairos	Hypervisor platform
Router 1	TP-Link TD-W8961N Ver:3.0	Compatibility research — not flashed
Router 2 (ISP)	Huawei B636-336 (Orange) @ 192.168.1.1	Upstream gateway — read-only
OpenWRT VM	256 MB RAM, 1 vCPU, 512 MB qcow2	Virtualised router/firewall/VPN

3.2 Software Stack

Software	Version	Purpose
OpenWRT	23.05.3	Router/firewall OS running in the VM
KVM	Kernel mod	Hardware-accelerated virtualisation engine
QEMU	System pkg	Hardware emulation (CPU, disks, NICs)
libvirt	Daemon	VM management API and lifecycle control
WireGuard	kmod	VPN protocol — Linux kernel module
Suricata	7.0.3	Intrusion Detection System
ET Open rules	48,867 sigs	Community-maintained Suricata ruleset
Nmap	Host pkg	Network and port scanner
Wireshark	Host pkg	GUI packet capture and analysis

3.3 Network Topology

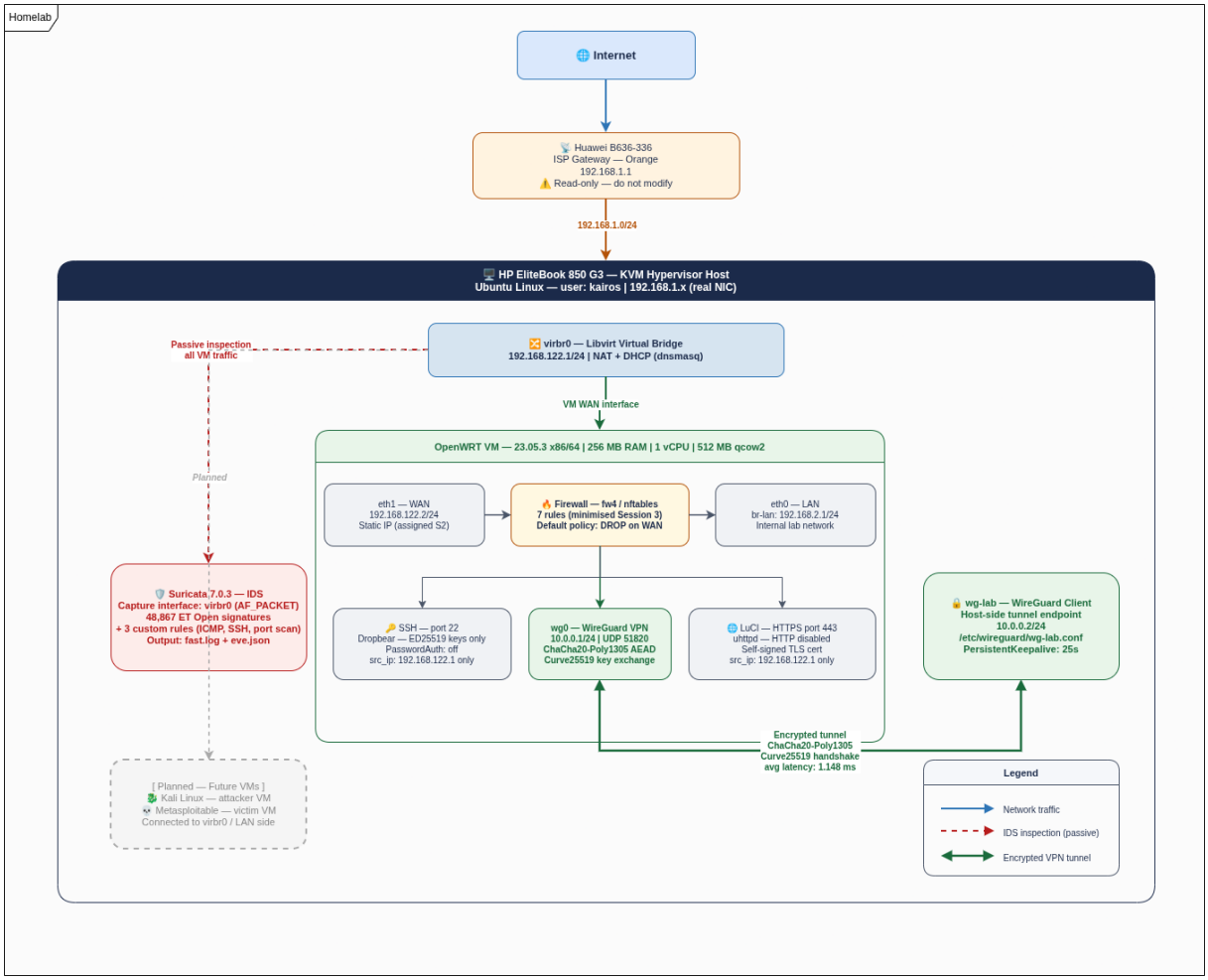

Figure 1: Full Lab Network Topology

3.4 IP Address Schema

Network	Subnet	Gateway	Purpose
Real home network	192.168.1.0/24	192.168.1.1	Upstream ISP — read-only
KVM virbr0 bridge	192.168.122.0/24	192.168.122.1	VM WAN network (libvirt)
OpenWRT WAN (eth1)	192.168.122.2/32	192.168.122.1	Static WAN (assigned S2)
OpenWRT LAN (br-lan)	192.168.2.0/24	192.168.2.1	Internal lab network
WireGuard tunnel	10.0.0.0/24	N/A	Encrypted management
Host VPN IP	10.0.0.2/32	—	Host VPN endpoint

4. Session 1 — KVM Setup & OpenWRT VM Deployment

Date	Host	Goals
March 5, 2026	HP EliteBook 850 G3	Hardware research, KVM install, OpenWRT VM first boot

4.1 Hardware Compatibility Research

The TP-Link TD-W8961N Ver:3.0 was checked against the OpenWRT Table of Hardware at openwrt.org/toh/start.

```
# Result: No entry found for TD-W8961N Ver:3.0
```

Subsystem	Function	OpenWRT Support
DSL Modem Chip	Connects to ADSL phone line	**No** — proprietary, no open-source drivers
Router/WiFi SoC	Runs OS, firewall, NAT, WiFi	**Often yes** — common SoC chips
Combined result	Both chips needed together	**No** — DSL broken = no internet

DANGER: DO NOT FLASH

Flashing unsupported firmware = near-certain permanent brick. No serial console access = unrecoverable without hardware soldering. This path was rejected.

4.2 KVM Virtualisation Stack

Component	Package	Role	Analogy
KVM	qemu-kvm	Hardware-accelerated virtualisation	The engine
QEMU	qemu-kvm	Emulates hardware for each VM	The chassis
libvirt	libvirt-daemon-system	Management API layer	The dashboard
virt-manager	virt-manager	GUI VM management	Steering wheel
virtinst	virtinst	CLI tools (virt-install)	The toolkit
bridge-utils	bridge-utils	VM network interface sharing	Network switchboard

4.2.1 Step 1 — CPU Virtualisation Verification

```
$ egrep -c '(vmx|svm)' /proc/cpuinfo
8

$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used
```

SUCCESS

8 threads with Intel VT-x (vmx) confirmed. Hardware-accelerated virtualisation available.

4.2.2 Step 2 — Install KVM Stack

```
sudo apt update && sudo apt install -y \
  qemu-kvm libvirt-daemon-system libvirt-clients \
  virt-manager virtinst bridge-utils virt-viewer
```

4.2.3 Step 3 — User Group Configuration

```
sudo usermod -aG libvirt kairos
sudo usermod -aG kvm kairos

$ groups kairos
```

```
kairos : kairos adm cdrom sudo kvm docker libvirt
```

WARNING: Log Out Required

Group membership changes require a full desktop logout and re-login. A new terminal window alone is not sufficient.

4.2.4 Step 4 — Start libvirtd Service

```
sudo systemctl enable --now libvirtd
sudo systemctl status libvirtd
# Active: active (running)
# dnsmasq running on virbr0 -- DHCP ready for VMs
```

4.2.5 Step 5 — Verify KVM Stack

```
$ virsh list --all
 Id   Name   State
--------------------
(empty -- correct, no VMs created yet)
```

4.3 OpenWRT Image Preparation

4.3.1 Step 6 — Image Selection

Choice	Value	Reason
Architecture	x86/64	KVM virtualises x86 hardware
Image format	ext4-combined	Single .img file: kernel + root filesystem
Version	23.05.3	Current stable LTS release
File format	.img.gz	Compressed disk image — must decompress first

4.3.2 Step 7 — Download & SHA256 Verification

```
mkdir -p ~/homelab/openwrt && cd ~/homelab/openwrt

wget https://downloads.openwrt.org/releases/23.05.3/targets/x86/64/\
openwrt-23.05.3-x86-64-generic-ext4-combined.img.gz

wget https://downloads.openwrt.org/releases/23.05.3/targets/x86/64/sha256sums

sha256sum -c sha256sums --ignore-missing
# openwrt-23.05.3-x86-64-generic-ext4-combined.img.gz: OK
```

TIP: SHA256 Verification is Non-Negotiable

A FAILED result means the file is corrupted or tampered with. Never use a file that fails checksum verification.

4.3.3 Step 8 — Decompress & Convert to qcow2

```
gunzip openwrt-23.05.3-x86-64-generic-ext4-combined.img.gz
# "trailing garbage ignored" warning is harmless

qemu-img convert -f raw -O qcow2 \
  openwrt-23.05.3-x86-64-generic-ext4-combined.img \
  openwrt.qcow2

qemu-img info openwrt.qcow2
# file format: qcow2
# virtual size: 512 MiB   (allocated virtual space)
# disk size: 23.4 MiB     (only 23 MB actual disk -- sparse!)
```

Format	Snapshots	KVM	Thin Prov.	Notes
raw	No	No	No	Triggers QEMU format detection warnings
qcow2	Yes	Yes	Yes	Recommended for all KVM VMs

4.4 VM Creation — Issue Log

4.4.1 Issue 1 — libvirt Default Network Not Found

ERROR

```
Network not found: no network with matching name 'default'
```

```
virsh net-list --all
# Returns: empty -- but:

virsh net-start default
# Returns: "already active" -- contradiction!

ip addr show virbr0
# virbr0 EXISTS with 192.168.122.1/24 assigned
# Conclusion: network IS running at OS level; virsh DB out of sync
```

Root Cause. The libvirt default NAT network existed in the database but had never been started. The `virsh` display was out of sync with the actual OS-level network state.

FIX APPLIED

Re-created the default network XML definition from scratch and registered it with `virsh net-define`. The virbr0 bridge was already present, the fix re-synced libvirt's internal database.

4.4.2 Issue 2 — QEMU Bridge ACL File Missing

ERROR

```
stderr=failed to parse default acl file '/etc/qemu/bridge.conf'
```

Root Cause. QEMU's bridge helper requires an explicit whitelist of bridges it is permitted to use. The `/etc/qemu/` directory and `bridge.conf` file did not exist. QEMU refuses to proceed rather than assume open access, secure-by-default: fail closed.

CONCEPT: ACL Whitelisting

Rather than allowing QEMU to use any bridge on the system, it requires an explicit allow list. This limits blast radius if QEMU were ever exploited.

FIX APPLIED

```
sudo mkdir -p /etc/qemu
echo 'allow virbr0' | sudo tee /etc/qemu/bridge.conf
sudo chmod 644 /etc/qemu/bridge.conf
```

4.4.3 Issue 3 — bridge.conf Permission Denied

ERROR

```
cat: /etc/qemu/bridge.conf: Permission denied
```

Root Cause. File was created with permissions 640. The `qemu-bridge-helper` process runs as a different effective user and could not read it. The file contains no secrets, so world-readable is appropriate.

FIX APPLIED

```
sudo chmod 644 /etc/qemu/bridge.conf
```

4.4.4 Issue 4 — qemu-bridge-helper Missing setuid Bit

ERROR

```
stderr=failed to create tun device: Operation not permitted
```

Root Cause. Creating TUN/TAP devices requires elevated privileges. `qemu-bridge-helper` is designed as a privileged helper, but its setuid bit was not set, so it ran as calling user `kairos` , insufficient for TUN/TAP creation.

CONCEPT: setuid Binaries

The setuid bit (`chmod u+s`) causes a binary to run with the privileges of its owner (root) regardless of who executes it. This allows unprivileged users to perform specific privileged operations through a controlled, minimal binary.

FIX APPLIED

```
# Before: -rwxr-xr-x  (runs as calling user)
# After:  -rwsr-xr-x  (runs as binary owner = root)
sudo chmod u+s /usr/lib/qemu/qemu-bridge-helper
```

4.4.5 Issue 5 — IP Conflict: OpenWRT LAN vs Huawei Router

Problem. OpenWRT's default LAN IP is `192.168.1.1`. The Huawei home router is also `192.168.1.1`. Two devices sharing the same IP on connected networks causes routing ambiguity.

FIX APPLIED

```
uci set network.lan.ipaddr='192.168.2.1'
uci commit network && service network restart

ip addr show br-lan
# inet 192.168.2.1/24  <-- conflict resolved
```

CONCEPT: UCI Configuration

UCI (Unified Configuration Interface) is OpenWRT's config management system. All settings live in `/etc/config/`. `uci commit` writes changes to disk; `service restart` applies them to the running system.

4.5 OpenWRT First Boot

4.5.1 VM Specifications

Parameter	Value	Rationale
RAM	256 MB	Typical usage 80–120 MB; 53%+ headroom
vCPUs	1	Sufficient for all router-class workloads
Disk	512 MB qcow2	Base system ≈120 MB; sparse: only 23 MB actual
NIC 1 (eth0)	virbr0	LAN side — enslaved into br-lan bridge
NIC 2 (eth1)	virbr0	WAN side — DHCP from libvirt dnsmasq
Graphics	VNC	Console access for initial headless setup

4.5.2 Security — Root Password

DANGER: CRITICAL FIRST ACTION

OpenWRT ships with NO root password. Anyone who can reach the device has full root access with zero authentication. This was the very first action after first boot.

```
passwd
# passwd: password for root changed by root
```

4.6 Session 1 Snapshots

```
virsh snapshot-create-as openwrt "fresh-install" \
  --description "Clean OpenWRT boot, no configuration"

virsh snapshot-create-as openwrt "base-config" \
  --description "Root password set, LAN=192.168.2.1, WAN on virbr0"

virsh snapshot-list openwrt
#  Name           Creation Time               State
# ----------------------------------------------------
#  fresh-install  2026-03-05 02:19:12 +0000   running
#  base-config    2026-03-05 02:44:09 +0000   running
```

5. Session 2 — Recovery, LuCI, SSH Hardening & Nmap

Date	Host	Goals
March 6, 2026	HP EliteBook 850 G3	VM recovery, LuCI access, SSH keys, Nmap scan, IP restriction

5.1 Issue 6 — VM Definition Lost After Host Reboot

Session 2 opened with the OpenWRT VM completely absent from libvirt.

```
$ virsh start openwrt
error: failed to get domain 'openwrt'

$ virsh list --all
 Id   Name   State
--------------------
(empty)
```

Root Cause. `virt-install` creates *transient* (in-memory only) VM definitions by default. These disappear on host reboot. The disk image was completely safe on disk.

```
$ ls -lh ~/homelab/openwrt/
-rw-r--r-- 1 kairos kairos 158M openwrt.qcow2  <-- safe, all config intact
```

FIX APPLIED

Re-ran `virt-install` with the existing qcow2 image. Immediately encountered Issue 7 below.

5.2 Issue 7 — Image Permission Denied

ERROR

```
Cannot access storage file '/home/kairos/homelab/openwrt/openwrt.qcow2'
(as uid:64055, gid:993):  Permission denied
```

Root Cause. QEMU runs as the unprivileged system user `libvirt-qemu` (uid:64055) for security. Linux home directories are private by default, `libvirt-qemu` cannot read files stored in `/home/kairos`.

CONCEPT: QEMU Privilege Separation

QEMU deliberately drops root privileges. If QEMU is ever exploited, an attacker gets `libvirt-qemu` privileges, not root. VM images must therefore live in system directories, not user home directories.

FIX APPLIED

```
sudo mv /home/kairos/homelab/openwrt/openwrt.qcow2 \
  /var/lib/libvirt/images/openwrt.qcow2

$ virsh list --all
 Id   Name      State
-------------------------
 1    openwrt   running     <-- VM running, all S1 config intact
```

5.3 Internet Connectivity Verification

```
root@OpenWrt:/# ping -c 4 8.8.8.8
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 45.665/50.995/60.492 ms

root@OpenWrt:/# ping -c 4 google.com
4 packets transmitted, 4 packets received, 0% packet loss
```

Test	Result	What It Confirms
ping 8.8.8.8	0% loss	Raw IP routing — WAN to internet path functional
ping google.com	0% loss	DNS resolution working

5.4 LuCI Web Interface Access

5.4.1 Issue 8a — DHCP IP Changed Between Sessions

```
# Session 1 WAN IP: 192.168.122.120  (DHCP)
# Session 2 WAN IP: 192.168.122.175  (new DHCP lease -- different!)
```

```
uci set network.wan.proto='static'
uci set network.wan.ipaddr='192.168.122.2'
uci set network.wan.netmask='255.255.255.0'
uci set network.wan.gateway='192.168.122.1'
uci set network.wan.dns='8.8.8.8'
uci commit network && service network restart
```

5.4.2 Issue 8b — RFC 1918 Filter Blocking LuCI

Fix: Assign Static WAN IP. Browser showed `ERR_ADDRESS_UNREACHABLE` even with the correct static IP.

```
uci show uhttpd | grep rfc
uhttpd.main.rfc1918_filter='1'  <-- THIS was blocking access
```

Root Cause. In production, the WAN IP is a public internet address and this filter prevents private-range requests reaching the admin panel. In this lab, the "WAN" interface is virbr0 (a private bridge), so the filter incorrectly blocks the legitimate host IP.

FIX APPLIED

```
uci set uhttpd.main.rfc1918_filter='0'
uci set uhttpd.main.listen_http=''   # disable HTTP port 80 entirely
uci commit uhttpd && service uhttpd restart
```

SUCCESS

LuCI loaded at `https://192.168.122.2`. Static IP confirmed. HTTP port 80 disabled.

5.5 SSH Key Authentication Setup

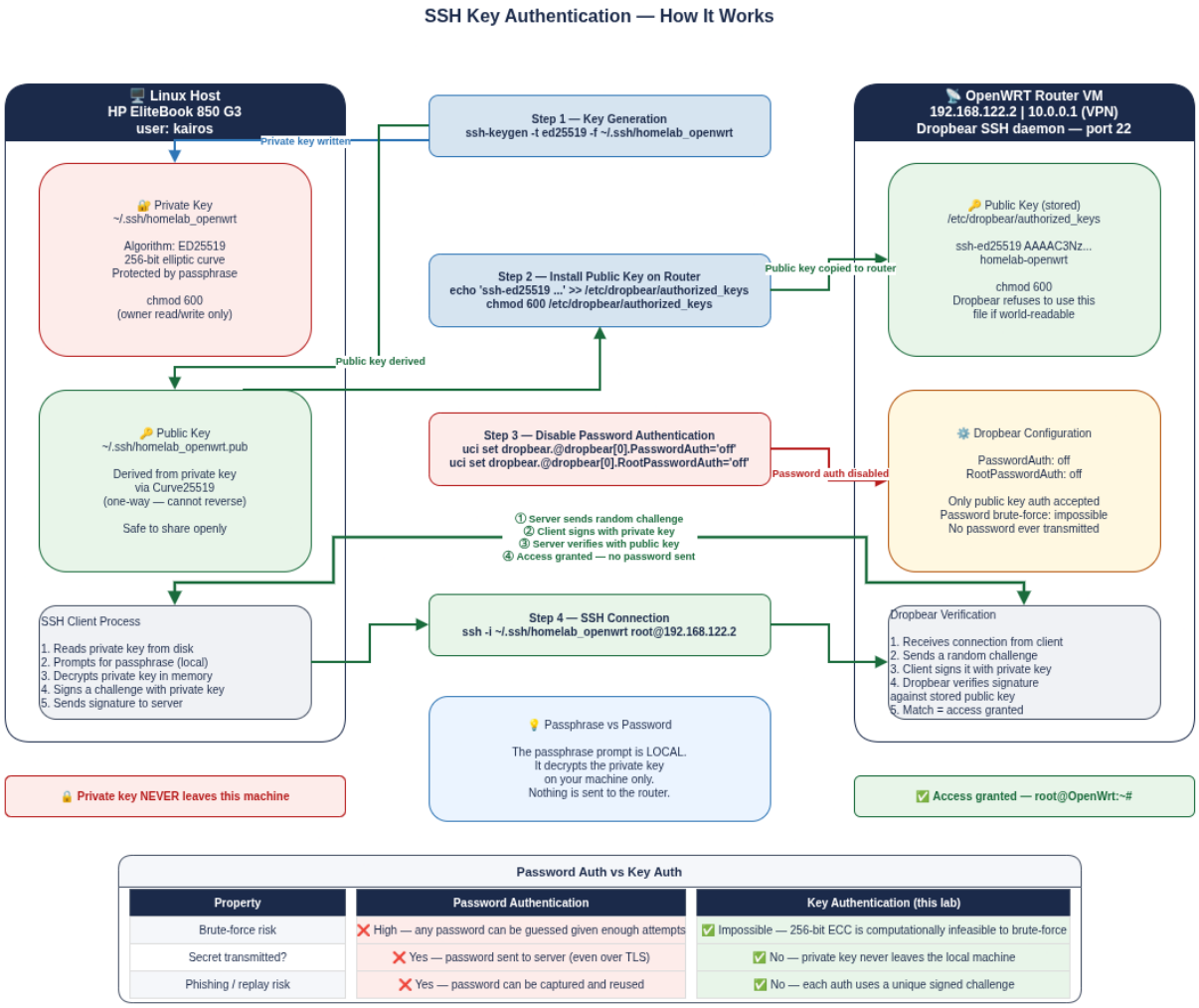

Figure 2: SSH Key Authentication Flow

5.5.1 Step 1 — Generate ED25519 Key Pair

```
ssh-keygen -t ed25519 -C 'homelab-openwrt' -f ~/.ssh/homelab_openwrt

# Creates:
# ~/.ssh/homelab_openwrt      <-- PRIVATE KEY (never share, never transmit)
# ~/.ssh/homelab_openwrt.pub  <-- PUBLIC KEY  (copied to router)
```

Algorithm	Key Size	Security	Speed	Recommendation
RSA	2048–4096 bit	Good	Slower	Legacy — still supported
ED25519	256 bit (ECC)	Excellent	Fast	**Best practice**

5.5.2 Step 2 — Install Public Key on OpenWRT

```
# Inside OpenWRT console:
mkdir -p /etc/dropbear
echo 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... homelab-openwrt' \
  >> /etc/dropbear/authorized_keys
chmod 600 /etc/dropbear/authorized_keys
# Dropbear REFUSES authorized_keys if world-readable (security check)
```

5.5.3 Step 3 — Add Firewall Rule to Allow SSH from WAN

```
uci add firewall rule
uci set firewall.@rule[-1].name='Allow-SSH-from-host'
```

```
uci set firewall.@rule[-1].src='wan'
uci set firewall.@rule[-1].dest_port='22'
uci set firewall.@rule[-1].proto='tcp'
uci set firewall.@rule[-1].target='ACCEPT'
uci commit firewall && service firewall restart
```

5.5.4 Step 4 — Test SSH Key Login

```
$ ssh -i ~/.ssh/homelab_openwrt root@192.168.122.2
Enter passphrase for key '/home/kairos/.ssh/homelab_openwrt':
BusyBox v1.36.1 ... OpenWrt 23.05.3
root@OpenWrt:~#   <-- logged in via key, no password sent to router
```

5.5.5 Step 5 — Disable Password Authentication

```
uci set dropbear.@dropbear[0].PasswordAuth='off'
uci set dropbear.@dropbear[0].RootPasswordAuth='off'
uci commit dropbear && service dropbear restart
```

5.5.6 Step 6 — Verify Password Login Rejected

```
$ ssh -o PreferredAuthentications=password \
      -o PubkeyAuthentication=no root@192.168.122.2
root@192.168.122.2: Permission denied (publickey).
# 'publickey' in error = server only accepts keys -- correct
```

SUCCESS

Password brute-force attacks are now impossible, there is no password authentication path to attack.

5.6 Nmap Attack Surface Analysis

5.6.1 Full Port Scan

```
nmap -sV -sC -p- --open 192.168.122.2
# -sV    Version detection
# -sC    Default scripts (TLS cert, SSH fingerprint)
# -p-    All 65,535 TCP ports
# --open Show only open ports
# Duration: ~275 seconds (4.5 minutes)
```

5.6.2 Scan Results

```
PORT      STATE  SERVICE   VERSION
22/tcp    open   ssh       Dropbear sshd (protocol 2.0)
443/tcp   open   ssl/https
| ssl-cert: commonName=OpenWrt / organizationName=OpenWrtd6372c39
|           stateOrProvinceName=Somewhere / countryName=ZZ
|           Not valid before: 2026-03-05T02:17:44
|_          Not valid after:  2028-03-04T02:17:44

Not shown: 63594 filtered, 1939 closed
Nmap done: 1 IP address scanned in 275.13 seconds
```

Port / Range	State	Risk	Analysis
22/tcp	Open	Low	Password auth disabled, keys only. Restrict by IP next.
443/tcp	Open	Medium	Admin panel on WAN. Self-signed cert reveals OpenWRT identity.
63,594 pts	Filtered	Good	Firewall silent drop, attacker gets no information.
1,939 pts	Closed	Good	No service running.

5.6.3 TLS Certificate Fingerprinting

The Nmap script extracted the TLS certificate, revealing device identity information:

Field	Value	Security Implication
commonName	OpenWrt	Immediately identifies this as an OpenWRT device
organizationName	OpenWrtd6372c39	Default name — unique device fingerprint
stateOrProvinceName	Somewhere	Default placeholder — confirms unmodified cert
countryName	ZZ	OpenWRT default — another fingerprinting indicator
Valid from	2026-03-05	Reveals exact device setup date to any scanner

5.7 Firewall Hardening — Source IP Restriction

```
# Find rule indices
uci show firewall | grep -n 'LuCI\|SSH'
# 89:firewall.@rule[9].name='Allow-LuCI-from-host'
# 95:firewall.@rule[10].name='Allow-SSH-from-host'

# Restrict both to host IP only
uci set firewall.@rule[9].src_ip='192.168.122.1'
uci set firewall.@rule[10].src_ip='192.168.122.1'
uci commit firewall && service firewall restart
```

5.7.1 Verification Scan

```
$ nmap -sV -p 22,80,443 192.168.122.2
PORT    STATE   SERVICE
22/tcp  open    ssh
80/tcp  closed  http     <-- disabled successfully
443/tcp open    ssl/https
```

5.8 VM Persistence Fix

```
virsh define <(virsh dumpxml openwrt)
# Domain 'openwrt' defined from /dev/fd/63

virsh dominfo openwrt | grep Persistent
# Persistent:     yes

virsh autostart --disable openwrt
# Autostart: disable  (manual start preferred for lab control)
```

5.9 Session 2 Snapshots

```
virsh snapshot-create-as openwrt "luci-accessible" \
  --description "Static WAN 192.168.122.2, LuCI working, rfc1918 off"

virsh snapshot-create-as openwrt "hardened-phase4" \
  --description "SSH keys only, LuCI+SSH restricted to host IP, HTTP disabled"
```

5.10 Session 2 Hardening Summary

Control	Status	Detail
Root password		Set on first boot (Session 1)
SSH key auth		ED25519, passphrase protected
SSH password login	Disabled	PasswordAuth=off
SSH access restriction		src_ip=192.168.122.1 only
LuCI port 80	Disabled	listen_http="
LuCI access restriction		src_ip=192.168.122.1 only
Static WAN IP		192.168.122.2 — no DHCP drift
VM persistence		virsh define — survives reboots
Attack surface	2 open ports	Out of 65,535 scanned

6. Session 3 — Firewall Audit & WireGuard VPN

Date	Host	Goals
March 6, 2026	HP EliteBook 850 G3	Firewall ruleset audit, WireGuard VPN, encrypted management

6.1 Session Startup

```
$ virsh start openwrt
Domain 'openwrt' started

$ ssh -i ~/.ssh/homelab_openwrt root@192.168.122.2
root@OpenWrt:~#   <-- instant access, two commands, no setup
```

6.2 Firewall Ruleset Audit

6.2.1 Rule-by-Rule Analysis

Rule	Direction/Port	Decision	Reason
Allow-DHCP-Renew	WAN port 68	Kept	Required for WAN IP assignment
Allow-Ping	WAN ICMP	Removed	Reveals router to scanners
Allow-IGMP	WAN IGMP	Removed	Multicast, not needed in VM lab
Allow-DHCPv6	WAN port 546	Kept	Required for IPv6
Allow-MLD	fe80::/10	Kept	IPv6 link-local, restricted scope
Allow-ICMPv6-Input	WAN ICMPv6	Kept	Required for IPv6
Allow-ICMPv6-Forward	WAN ICMPv6	Kept	Required for IPv6 forwarding
Allow-IPSec-ESP	WAN ESP	Removed	Using WireGuard, not IPSec
Allow-ISAKMP	WAN port 500	Removed	IPSec key exchange unneeded
Allow-LuCI-from-host	WAN port 443	Kept	Our rule, IP-restricted
Allow-SSH-from-host	WAN port 22	Kept	Our rule, IP-restricted

6.2.2 Deletion Process (Index-Aware)

WARNING: Index Shifting

Deleting a UCI rule renumbers all subsequent rules. Verify indices after each deletion by re-running: `uci show firewall | grep name`

```
# Step 1: Remove Allow-Ping (currently index 1)
uci delete firewall.@rule[1]
uci commit firewall && service firewall restart

# Step 2: Remove Allow-IGMP (now at index 1 after shift)
uci delete firewall.@rule[1]
uci commit firewall && service firewall restart

# Step 3: Remove Allow-IPSec-ESP (now at index 5)
uci delete firewall.@rule[5]
# Allow-ISAKMP shifts to index 5 -- remove immediately
uci delete firewall.@rule[5]
uci commit firewall && service firewall restart
```

6.2.3 Final Ruleset (7 rules from 11)

Index	Rule Name	Purpose
rule[0]	Allow-DHCP-Renew	Upstream IP assignment
rule[1]	Allow-DHCPv6	IPv6 address assignment
rule[2]	Allow-MLD	IPv6 link-local multicast
rule[3]	Allow-ICMPv6-Input	IPv6 input handling
rule[4]	Allow-ICMPv6-Forward	IPv6 forwarding
rule[5]	Allow-LuCI-from-host	Web admin : host only
rule[6]	Allow-SSH-from-host	SSH : host only

SUCCESS

7 rules from 11. 4 unnecessary rules eliminated. Attack surface reduced without breaking functionality.

6.3 WireGuard VPN Deployment

6.3.1 Protocol Comparison

Property	WireGuard	OpenVPN	IPSec
Lines of code	∼4,000	∼600,000	∼400,000
Linux kernel	Yes (since 5.6)	No (userspace)	Partial
Crypto suite	Fixed modern (safe)	Configurable (risky)	Complex
Setup	Simple	Complex	Very complex

6.3.2 WireGuard Cryptographic Stack

Package	Algorithm	Role
kmod-crypto-lib-chacha20	ChaCha20	Stream cipher, encrypts all tunnel data
kmod-crypto-lib-poly1305	Poly1305	MAC, detects any tampering
kmod-crypto-lib-chacha20poly1305	ChaCha20-Poly1305 AEAD	Encrypt + authenticate in one pass
kmod-crypto-lib-curve25519	Curve25519	Elliptic curve key exchange

6.3.3 Step 1 — Install on OpenWRT

```
opkg update
opkg install wireguard-tools kmod-wireguard luci-proto-wireguard
# Full crypto dependency chain resolved automatically
```

6.3.4 Step 2 — Key Generation (Both Sides)

```
# ON OPENWRT (server):
mkdir -p /etc/wireguard
wg genkey | tee /etc/wireguard/server_private.key | \
  wg pubkey > /etc/wireguard/server_public.key
chmod 600 /etc/wireguard/server_private.key

# Private key: (stays here)
# Public key:  (share)

# ON LINUX HOST (client):
mkdir -p ~/homelab/wireguard
wg genkey | tee ~/homelab/wireguard/client_private.key | \
  wg pubkey > ~/homelab/wireguard/client_public.key
chmod 600 ~/homelab/wireguard/client_private.key

# Private key: (stays here)
# Public key:  (share)
```

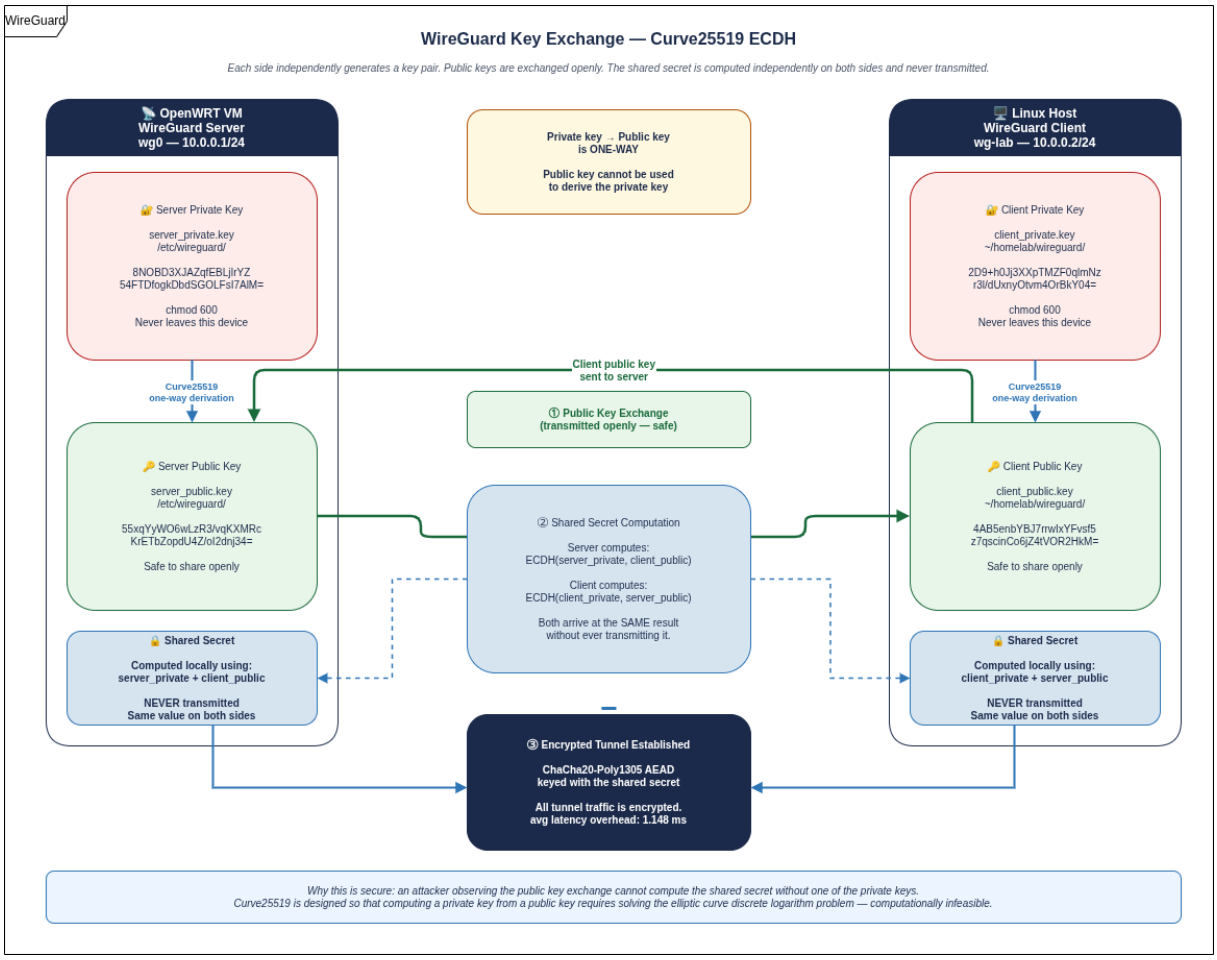

Figure 3: WireGuard Key Exchange — Curve25519 ECDH

6.3.5 Step 3 — Server Configuration (OpenWRT)

```
# Interface
uci set network.wg0=interface
uci set network.wg0.proto='wireguard'
uci set network.wg0.private_key='[redacted]'
uci set network.wg0.listen_port='51820'
uci set network.wg0.addresses='10.0.0.1/24'

# Peer registration (Linux host)
uci add network wireguard_wg0
uci set network.@wireguard_wg0[-1].public_key='[redacted]'
uci set network.@wireguard_wg0[-1].allowed_ips='10.0.0.2/32'
uci set network.@wireguard_wg0[-1].description='linux-host'
uci commit network && service network restart
```

6.3.6 Verify Server Side

```
root@OpenWrt:~# ip addr show wg0
6: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420
    inet 10.0.0.1/24 scope global wg0

root@OpenWrt:~# wg show
interface: wg0
  public key: [redacted]
  private key: (hidden)
  listening port: 51820
peer: [redacted]
  allowed ips: 10.0.0.2/32
```

6.3.7 Step 4 — Firewall VPN Zone

```
uci add firewall zone
uci set firewall.@zone[-1].name='vpn'
uci set firewall.@zone[-1].input='ACCEPT'
uci set firewall.@zone[-1].output='ACCEPT'
uci set firewall.@zone[-1].forward='REJECT'
uci set firewall.@zone[-1].network='wg0'

uci add firewall rule
uci set firewall.@rule[-1].name='Allow-WireGuard'
uci set firewall.@rule[-1].src='wan'
uci set firewall.@rule[-1].dest_port='51820'
uci set firewall.@rule[-1].proto='udp'
uci set firewall.@rule[-1].target='ACCEPT'
uci commit firewall && service firewall restart
```

6.3.8 Step 5 — Client Configuration (Linux Host)

```
sudo tee /etc/wireguard/wg-lab.conf << EOF
[Interface]
PrivateKey = [redacted]
Address = 10.0.0.2/24

[Peer]
PublicKey = [redacted]
Endpoint = 192.168.122.2:51820
AllowedIPs = 10.0.0.0/24
PersistentKeepalive = 25
EOF
sudo chmod 600 /etc/wireguard/wg-lab.conf
```

Setting	Value	Explanation
PrivateKey	client priv key	Linux host's cryptographic identity
Address	10.0.0.2/24	Host IP inside VPN tunnel
PublicKey	server pub key	Identifies OpenWRT as trusted server
Endpoint	192.168.122.2:51820	Where to send WireGuard packets
AllowedIPs	10.0.0.0/24	Split tunnel: only VPN subnet routed
PersistentKeepalive	25 sec	Prevents NAT table expiry

6.3.9 Step 6 — Tunnel Establishment & Testing

```
$ sudo wg-quick up wg-lab
[#] ip link add wg-lab type wireguard
[#] ip -4 address add 10.0.0.2/24 dev wg-lab
[#] ip link set mtu 1420 up dev wg-lab

$ sudo wg show
interface: wg-lab
  latest handshake: 10 seconds ago
  transfer: 92 B received, 180 B sent

$ ping -c 4 10.0.0.1
rtt min/avg/max/mdev = 0.812/1.148/1.848/0.411 ms
4 packets transmitted, 4 received, 0% packet loss

$ ssh -i ~/.ssh/homelab_openwrt root@10.0.0.1
root@OpenWrt:~#   <-- SSH through the encrypted tunnel
```

SUCCESS

0% packet loss. Average latency 1.148 ms, negligible encryption overhead. SSH management now flows through the ChaCha20-Poly1305 encrypted WireGuard tunnel.

6.4 Session 3 Snapshot

```
virsh snapshot-create-as openwrt "wireguard-vpn" \
```

```
  --description "WireGuard configured, tunnel tested, SSH over VPN working"
```

7. Session 4 — Suricata IDS & GitHub Publication

Date	Host	Goals
March 8, 2026	HP EliteBook 850 G3	Suricata IDS, custom rules, alert verification, GitHub publication

7.1 Session Startup — Three-Command Workflow

```
$ virsh start openwrt
$ sudo wg-quick up wg-lab
$ ssh -i ~/.ssh/homelab_openwrt root@10.0.0.1
# All three commands succeeded within 30 seconds.
```

7.2 Suricata IDS Overview

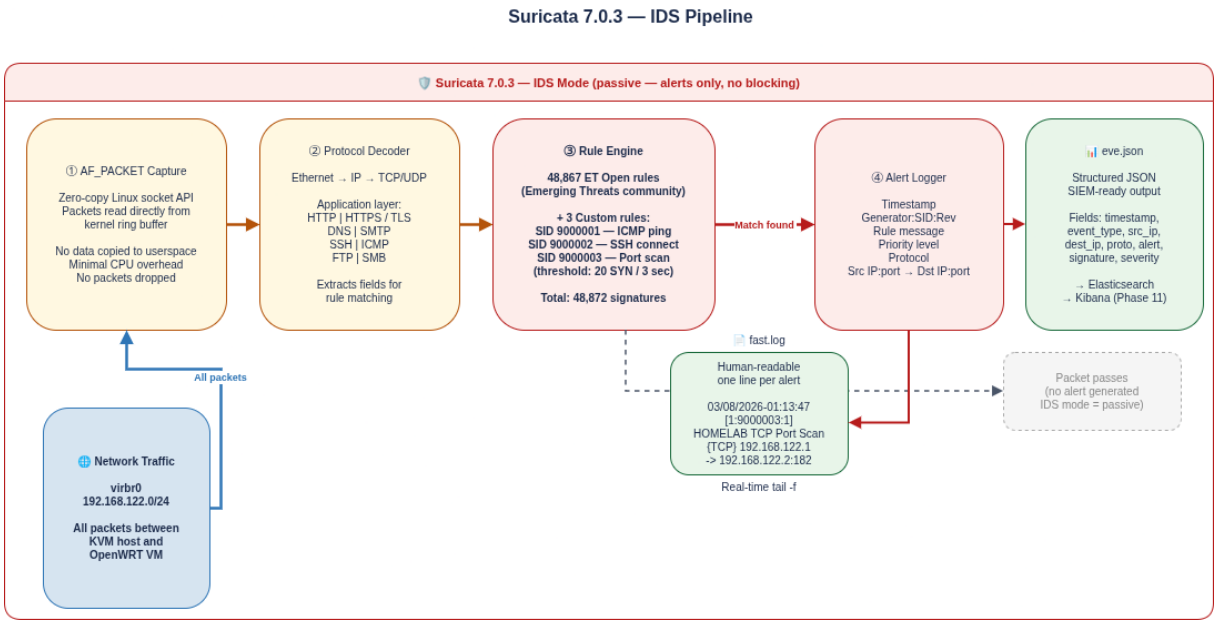

Figure 4: Suricata pipeline

Suricata is an open-source Intrusion Detection System (IDS) developed by the Open Information Security Foundation (OISF). It inspects network traffic in real time, matching packets against a ruleset of known attack signatures. In IDS mode it alerts only, it does not block traffic.

7.3 Suricata Setup

7.3.1 Step 1 — Update Emerging Threats Ruleset

```
sudo suricata-update
# Fetching emerging.rules.tar.gz from emergingthreats.net
# Enabled: 48867   Disabled: 14
# Testing with suricata -T... Done.
```

7.3.2 Step 2 — Configure Interface

```
# Edit /etc/suricata/suricata.yaml -- change capture interface:
# af-packet:
#   - interface: virbr0

sudo suricata -T -c /etc/suricata/suricata.yaml 2>&1 | tail -2
# 48872 signatures processed.
# Configuration provided was successfully loaded. Exiting.
```

7.4 Custom Rule Writing

7.4.1 Rule Anatomy

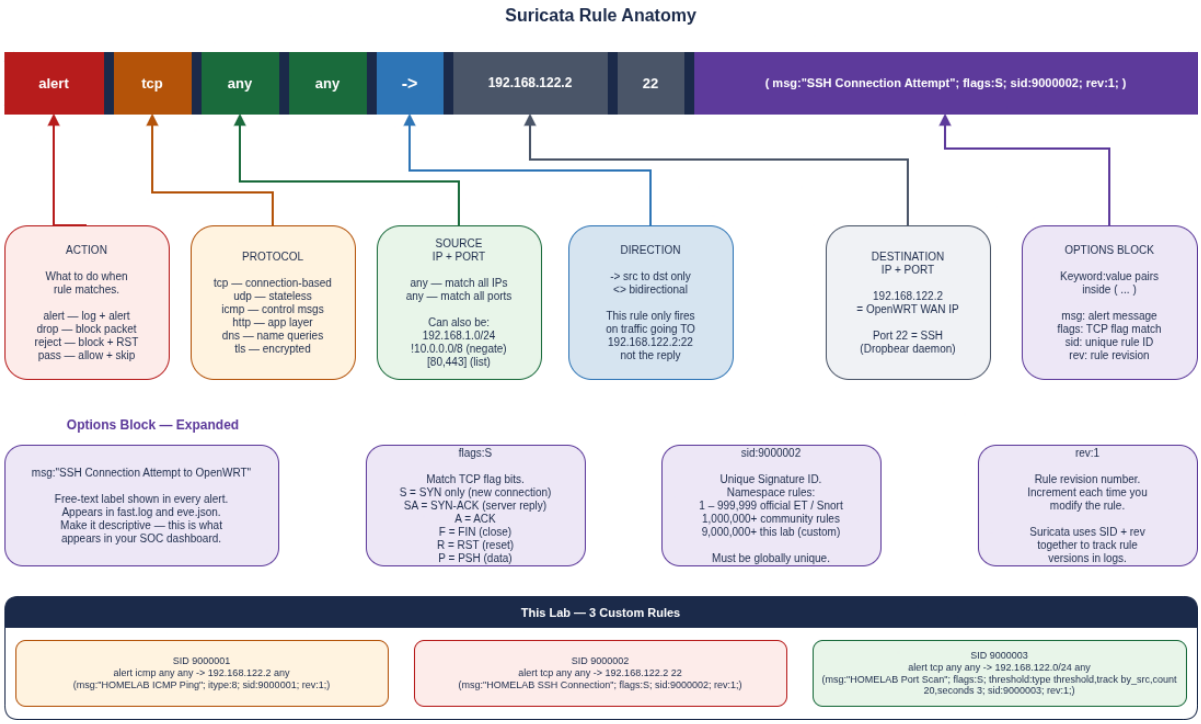

Figure 5: Suricata rule anatomy

7.4.2 Issue 9 — Custom Rules Not Loading

ERROR

Rules written to `/etc/suricata/rules/local.rules` — not loading on Suricata start.

Directory	Purpose	Correct for Custom Rules?
/etc/suricata/rules/	Config dir — example rules	No
/var/lib/suricata/rules/	Data dir — suricata-update writes here	Yes

FIX APPLIED

```
sudo cp /etc/suricata/rules/local.rules \
  /var/lib/suricata/rules/local.rules

# Add to /etc/suricata/suricata.yaml rule-files section:
# rule-files:
#   - suricata.rules
#   - local.rules    # exactly 2 spaces -- YAML is strict
```

WARNING: YAML Indentation Strictness

One space instead of two caused a silent rule load failure. Always verify config with `suricata -T` after editing.

7.4.3 The Three Custom Rules

```
# File: /var/lib/suricata/rules/local.rules

# Rule 1: Detect ICMP ping to OpenWRT WAN IP
alert icmp any any -> 192.168.122.2 any (
  msg:"HOMELAB ICMP Ping to OpenWRT";
  itype:8;          # echo request only (not replies)
  sid:9000001; rev:1;
)
```

```
# Rule 2: Detect every new SSH connection attempt
alert tcp any any -> 192.168.122.2 22 (
  msg:"HOMELAB SSH Connection Attempt to OpenWRT";
  flags:S;          # SYN flag only = new TCP connection
  sid:9000002; rev:1;
)

# Rule 3: Detect port scan (threshold-based)
alert tcp any any -> 192.168.122.0/24 any (
  msg:"HOMELAB TCP Port Scan Detected";
  flags:S;
  threshold:type threshold,track by_src,count 20,seconds 3;
  sid:9000003; rev:1;
)
```

SID	Rule	Key Options	What It Detects
9000001	ICMP Ping	itype:8 = echo request	Any ping to OpenWRT WAN IP
9000002	SSH Monitor	flags:S = SYN only	Every new connection to port 22
9000003	Port Scan Detect	threshold 20/3 sec	Nmap-style SYN scan reconnaissance

7.5 Alert Triggering & Verification

```
# Start Suricata
sudo suricata -c /etc/suricata/suricata.yaml -i virbr0 -D \
  --pidfile /var/run/suricata.pid
sleep 15  # wait for all 48,872 rules to load
sudo truncate -s 0 /var/log/suricata/fast.log  # clear old alerts

# Trigger the rules
ping -c 5 192.168.122.2                            # -> rule 9000001
ssh -i ~/.ssh/homelab_openwrt root@192.168.122.2 'echo test'  # -> 9000002
sudo nmap -sS -p 1-200 192.168.122.2               # -> rule 9000003
```

7.5.1 Alert Output in fast.log

```
03/08/2026-01:13:19 [1:9000002:1] HOMELAB SSH Connection Attempt to OpenWRT
[Priority: 3] {TCP} 192.168.122.1:33700 -> 192.168.122.2:22

03/08/2026-01:13:47 [1:9000003:1] HOMELAB TCP Port Scan Detected
[Priority: 3] {TCP} 192.168.122.1:55233 -> 192.168.122.2:182

03/08/2026-01:13:47 [1:9000003:1] HOMELAB TCP Port Scan Detected
[Priority: 3] {TCP} 192.168.122.1:55233 -> 192.168.122.2:122
# ... 11 more port scan alerts
```

7.5.2 Why Rule 9000001 (ICMP) Did Not Fire

OpenWRT's firewall dropped the ICMP packets at the WAN interface before they reached Suricata's capture point on virbr0.

SUCCESS

Rule not firing = firewall is working correctly. The Allow-Ping rule removed in Session 3 remains effective. This outcome validates the Session 3 hardening.

7.6 GitHub Repository Publication

```
cd ~/homelab/homelab-openwrt-security
git add .
git commit -m "Initial lab documentation -- OpenWRT security homelab complete"
git remote set-url origin git@github.com:oeisthename/homelab-openwrt-security.git
sudo git push origin main
# 8 files changed, 345 insertions(+)
# 1f5de94..912f1ee  main -> main
```

WARNING: Key Sanitisation — Non-Negotiable

Private keys must NEVER be committed to public repositories. WireGuard configs were published with `REDACTED_REPLACE_WITH_YOUR_KEY` placeholders only.

7.7 Session 4 Snapshot

```
virsh snapshot-create-as openwrt "phase6-monitoring" \
  --description "Suricata deployed, 48872 sigs + 3 custom, GitHub published"
```

8. Complete Issues & Troubleshooting Log

#	S	Issue	Root Cause	Fix Applied	Concept
1	1	TD-W8961N not in OpenWRT ToH	ADSL chipset proprietary; flashing = brick	Pivoted to KVM virtualisation	Hardware risk assessment
2	1	libvirt network not found despite virbr0 present	virsh DB out of sync with OS state	Re-created network XML; virsh net-define	libvirt state management
3	1	/etc/qemu/bridge.conf missing	Secure-by-default: QEMU requires explicit ACL whitelist	Created bridge.conf; chmod 644	ACL whitelisting
4	1	TUN creation fails	setuid bit not set on qemu-bridge-helper	chmod u+s qemu-bridge-helper	setuid binaries, PoLP
5	1	IP conflict LAN 192.168.1.1 = Huawei	Same IP on two devices = routing ambiguity	uci set LAN to 192.168.2.1	IP schema planning
6	2	VM definition lost after reboot	virt-install creates transient (in-memory) definitions	virsh define; move image to /var/lib/libvirt/images/	Transient vs persistent definitions
7	2	LuCI inaccessible	rfc1918_filter=1 blocks private IPs on WAN	uci set rfc1918_filter=0	Context-aware security controls
8	4	Suricata rules not loading	Wrong directory; YAML indentation error (1 space vs 2)	Copied to /var/lib/suricata/rules/; fixed indentation	File path hierarchy; YAML strictness

9. Security Controls — Final State

Control	Implementation	S	Status
Root password	Strong password, first boot	S1	
SSH key auth	ED25519, passphrase protected	S2	
SSH password auth	PasswordAuth=off	S2	Disabled
SSH access restriction	src_ip=192.168.122.1 only	S2	
LuCI port 80	listen_http=”	S2	Disabled
LuCI access restriction	src_ip=192.168.122.1 only	S2	
Firewall minimised	11 rules → 7 rules	S3	
Ping disabled	Allow-Ping rule removed	S3	
Encrypted management	All admin via WireGuard VPN	S3	
IDS monitoring	Suricata 7.0.3, 48,872 signatures	S4	
Attack surface	2 open TCP ports / 65,535 scanned	S2	
Snapshot recovery	6 named rollback points	S1–4	

9.1 Attack Surface Evolution

Milestone	Open Ports	Key Change
Fresh install (S1)	∼10+ defaults	No hardening
After hardening (S2)	2 (22, 443)	Root password, SSH keys, IP restrictions, HTTP disabled
After audit (S3)	2 (22, 443)	4 rules removed; WireGuard invisible to TCP scanners
Final state (S4)	2 (22, 443)	All admin encrypted; Suricata monitoring active

10. Performance Metrics

Metric	Value	Notes
WireGuard latency avg	1.148 ms	RTT through encrypted tunnel (4-ping test)
WireGuard latency min	0.812 ms	Minimum observed
WireGuard latency max	1.848 ms	Maximum observed
Packet loss	0%	4/4 ICMP through tunnel
Full Nmap scan (65,535)	275 seconds	2 open, 63,594 filtered, 1,939 closed
ET rules loaded	48,867	Emerging Threats Open
Custom rules	3	ICMP, SSH, port scan
VM RAM usage	80–120 MB	Typical; 256 MB allocated
VM disk actual	23 MB	qcow2 sparse at creation

11. Cost Analysis

Item	Cost	Notes
HP EliteBook 850 G3	Already owned	~$150–200 USD refurbished
Additional hardware	$0	Zero new hardware required
OpenWRT, KVM, QEMU, libvirt	$0	GPL open source
WireGuard	$0	GPL (Linux kernel 5.6+)
Suricata + ET rules	$0	GPL (OISF), community free tier
All other tools	$0	Open source
TOTAL	**$0**	

12. Snapshot History

S	Snapshot Name	State Captured
S1	fresh-install	Clean first boot — zero configuration
S1	base-config	Root password set, LAN=192.168.2.1, VM issues resolved
S2	luci-accessible	Static WAN IP, LuCI accessible, rfc1918 off, VM persistent
S2	hardened-phase4	SSH keys, password auth disabled, IP restrictions, HTTP off
S3	wireguard-vpn	Firewall minimised, WireGuard operational, SSH over VPN
S4	phase6-monitoring	Suricata IDS active, 48,872 sigs + 3 custom, GitHub published

13. Skills & Competencies Demonstrated

Domain	Skills
Virtualisation	KVM/QEMU setup and management, VM lifecycle (create/snapshot/restore), qcow2 thin provisioning, libvirt networking, persistent VM definitions
Networking	IPv4/IPv6 addressing and subnetting, static IP config, bridge networking (virbr0, br-lan), NAT traversal, IP conflict diagnosis
VPN Engineering	WireGuard protocol design, Curve25519 key generation, encrypted tunnel establishment, split tunneling, latency measurement
Firewall Engineering	Zone-based firewalling (nftables/fw4), attack surface quantification, ruleset audit, UCI index management, source IP restriction
Security Hardening	SSH key cryptography (ED25519), password auth elimination, TLS cert fingerprinting, setuid management, service exposure minimisation
Intrusion Detection	Suricata IDS architecture, ET ruleset management, custom rule authoring (threshold, itype, flags), alert format analysis
Linux Administration	UCI config management, systemd service control, file permissions and setuid, apt and opkg package management
Portfolio	Git repository design, structured troubleshooting logs, sanitised config publishing, audience-aware documentation

14. Quick Reference Commands

14.1 Lab Start / Stop

Action	Command
Start VM	`virsh start openwrt`
SSH direct (no VPN)	`ssh -i ~/.ssh/homelab_openwrt root@192.168.122.2`
Bring VPN up	`sudo wg-quick up wg-lab`
SSH over VPN (S3+)	`ssh -i ~/.ssh/homelab_openwrt root@10.0.0.1`
Bring VPN down	`sudo wg-quick down wg-lab`
Shutdown VM	`virsh shutdown openwrt`

14.2 VM Management

Action	Command
List all VMs	`virsh list -all`
Connect to console	`virsh console openwrt`
Create snapshot	`virsh snapshot-create-as openwrt "name" -description "text"`
List snapshots	`virsh snapshot-list openwrt`
Revert to snapshot	`virsh snapshot-revert openwrt <name>`
Make definition persist	`virsh define <(virsh dumpxml openwrt)`

14.3 OpenWRT UCI

Action	Command
Show network config	`uci show network`
Show firewall rules	`uci show firewall | grep -E 'name|target|src|dest'`
Commit changes	`uci commit <section>`
Restart service	`service <name> restart`
Delete firewall rule	`uci delete firewall.@rule[N]`
List packages	`opkg list-installed`

14.4 WireGuard

Action	Command
Tunnel status	`sudo wg show`
Bring tunnel up/down	`sudo wg-quick up|down wg-lab`
Generate key pair	`wg genkey | tee priv.key | wg pubkey > pub.key`

14.5 Suricata IDS

Action	Command
Start (IDS mode)	`sudo suricata -c /etc/suricata/suricata.yaml -i virbr0 -D`
Watch alerts live	`sudo tail -f /var/log/suricata/fast.log`
Update ET rules	`sudo suricata-update`
Validate config	`sudo suricata -T -c /etc/suricata/suricata.yaml`
Clear alert log	`sudo truncate -s 0 /var/log/suricata/fast.log`

15. Future Roadmap

Phase	Planned Work	Skills Gained
Phase 8	Deploy Kali Linux attack VM on lab LAN	Offensive security, pen testing basics
Phase 9	Deploy Metasploitable victim VM	Vulnerability exploitation, CVE analysis
Phase 10	Write Suricata rules for specific Kali tool sigs	Signature writing from threat intelligence
Phase 11	SIEM: forward eve.json to Elasticsearch + Kibana	Log aggregation, threat hunting
Phase 12	VLAN segmentation on OpenWRT	Network segmentation, 802.1Q
Phase 13	Ansible playbooks for OpenWRT hardening	Infrastructure as Code
Phase 14	Replace self-signed LuCI cert	PKI, TLS certificate lifecycle

OpenWRT Cybersecurity Homelab : Complete Engineering Documentation

kairos • Sessions 1–4 • March 2026

github.com/oeisthename/homelab-openwrt-security